

Technics SX-WSA1R

System Exclusive Reference

Acoustic Modeling Synthesizer Module

Operating system version 2.0

Contents

1	Overview	3
1.1	Applicability	3
1.2	Ports	3
1.3	Verification	3
1.4	Conventions	3
2	Message framing	4
2.1	Identifiers	4
2.2	Termination	4
2.3	Length limit	4
3	The Technics message set	5
3.1	Message families	5
3.2	The model triple	5
3.3	Short messages	5
3.4	Tempo	6
3.5	Parameter families	6
4	Bulk dump	7
4.1	Categories and what each transmits	7
4.2	Data message	8
4.3	Payload encoding	8
4.4	Checksum	8
4.5	Requesting a dump	9
4.6	Opening a transfer	9
4.7	Acknowledgement and retry	9
4.8	Continuation	9
4.9	Closing	9
4.10	Receiving	10
5	What a block contains	11
5.1	Signatures	11
5.2	Names	11
6	Parameter messages	12
6.1	Parameter area	12
6.2	Whole-area request	12
6.3	A third region	12

7 Universal System Exclusive	13
7.1 Recognised messages	13
7.2 Messages not recognised	13
8 Errors	14
8.1 The messages	14
8.2 ERROR 40 — wrong identification	14
8.3 ERROR 41 — reception	14
8.4 ERROR 42 — transmission	15
8.5 Interference from other traffic	15
9 SX-WSA1 and SX-WSA1R	16
10 Relationship to other Technics instruments	17
10.1 What is shared	17
10.2 What distinguishes the products	17
10.3 Dump regions differ	17
11 Scope of this edition	19

Chapter 1

Overview

The Technics SX-WSA1R exchanges data over MIDI System Exclusive. This reference describes the messages it sends and accepts: their byte layout, the bulk-dump transfer protocol, the universal messages it honours, and what it reports when a transfer fails.

1.1 Applicability

This reference describes operating system version **2.0**. An earlier version 1 exists; its behaviour may differ and is not covered.

1.2 Ports

The instrument has two sets of MIDI sockets, MIDI 1 and MIDI 2. Ordinary performance data is handled on both. **Bulk dump transfers use MIDI 1 only** — a dump offered on MIDI 2 is not received, and the instrument sends dumps only from MIDI 1.

1.3 Verification

The message layouts, the payload encoding and the checksum rule in this reference have been checked against a bulk dump captured from a real instrument: 2 883 messages carrying a SOUND and COMBINATION transfer. Of these, 2 876 carry a checksum, and all 2 876 verify under the rule given in chapter 3. Every transfer in the capture delivered exactly the byte count its header declared.

1.4 Conventions

Byte values are hexadecimal. A field written *nn* takes one of the values listed for it. Ellipses inside a message stand for a body whose length depends on the message.

Every message begins with F0 and ends with F7, as required by MIDI.

Real-time messages are safe inside a transfer

System Real Time bytes (F8–FF) may be interleaved anywhere inside a System Exclusive message without disturbing it. Conversely, while the instrument is receiving a System Exclusive message it ignores F8 TIMING CLOCK, FA START, FB CONTINUE and FC STOP for transport purposes.

Chapter 2

Message framing

2.1 Identifiers

The instrument accepts exactly two System Exclusive identifiers:

identifier	meaning
50	Matsushita (Technics)
7E	Universal Non-Real Time

A message carrying any other identifier is tracked to its F7 and discarded. No error is reported for it.

There is no device or unit number after the identifier

Many instruments place a device id immediately after the manufacturer identifier. The SX-WSA1R does not. The byte following 50 is already the message-family byte described in chapter 3. A unit field does appear later in some families, but it is not where the convention would put it.

2.2 Termination

A message ends at F7. If any other status byte in the range 80–F6 arrives while a message is open, the message is abandoned and its contents discarded.

2.3 Length limit

A complete message may not exceed 256 bytes counting F0 and F7, that is 254 bytes between them. A longer message is abandoned and, inside a bulk-dump session, reported as a reception error. Dumps sent by the instrument use the full 256 bytes for all but the last message of each transfer.

Chapter 3

The Technics message set

3.1 Message families

After F0 50, the third byte selects a message family. The instrument accepts fourteen values:

21 22 23 24 25 27 28 29 2A 2B 2C 2D 7E 7F

26 is not accepted. The SX-WSA1R's own product identifier is 2D; its outgoing bulk-dump data messages all begin F0 50 2D.

3.2 The model triple

The families 21, 22, 2B, 2C and 2D continue with three fixed bytes:

04 *nn* 11

where *nn* is 00 or 01. A message whose triple the instrument does not recognise is rejected with ERROR 40.

The middle byte is not a constant of the product

The instrument transmits 04 00 11 and accepts both 04 00 11 and 04 01 11. A bulk dump captured from a real instrument carries 04 01 11 throughout. The transmitted value is fixed — it is not derived from the keyboard/rack setting described in chapter 9 — so the middle byte does not distinguish the two models either. A receiver must accept both values and must not use the middle byte to identify anything.

3.3 Short messages

These messages are complete as shown. They form the control vocabulary of a bulk-dump session.

message	meaning
F0 50 23 7E F7	acknowledge
F0 50 24 7E F7	not acknowledged
F0 50 27 7E F7	end of category
F0 50 28 7E F7	end of dump
F0 50 29 7E F7	abort
F0 50 2A 7E F7	destination full
F0 50 21 04 <i>nn</i> 11 F7	enquiry
F0 50 22 04 <i>nn</i> 11 F7	enquiry reply

3.4 Tempo

The family 25 carries a tempo setting, in the range 40 to 300 beats per minute. A value outside that range is treated as 120.

See chapter 9 — the tempo message is one of two things the rack does not support.

3.5 Parameter families

The families 2B and 2C carry individual parameter traffic rather than bulk transfer. Each addresses a large parameter space, and neither is enumerated in this edition; see chapter 11.

Chapter 4

Bulk dump

Bulk dump transfers the instrument's internal data as System Exclusive. The instrument is silent for the duration, and the transfer uses the MIDI 1 sockets only.

4.1 Categories and what each transmits

The SYSEX BULK DUMP screen offers five categories. Each sends one or more data messages at fixed dump addresses.

category	address bytes	length bytes	length	blocks
SYSTEM, PART & MIDI	40 00 00	00 00 20	32	
	40 00 20	00 12 60	2 400	
SOUND	20 00 00	10 00 00	262 144	32
COMBINATION	50 00 00	00 06 00	768	
	50 06 00	05 40 00	90 112	16
SEQUENCER	60 00 00	00 18 00	3 072	
	60 18 00	01 70 00	30 720	
	62 08 00	set at send time		
TOTAL KEYBOARD	the four categories in the order above			

Lengths are in source bytes; on the wire each occupies twice as many, because of the payload encoding described below.

The *blocks* column is internal structure, not extra messages: SOUND sends one header covering all 262 144 bytes, which is 32 repeats of an 8 192-byte layout, and COMBINATION's second block one header covering 90 112 bytes as 16 repeats of 5 632. Those are the instrument's 32 sounds and 16 combinations.

TOTAL KEYBOARD runs the four categories one after another in the order listed, each closed by its own end-of-category message, with a single end-of-dump message after the last.

The SEQUENCER category is the only one whose final message has no fixed length — it carries as much recorded material as the instrument holds.

The dump address is not a memory address

Within one category the addresses are consecutive: a block's address is the previous block's address plus its length. Across categories they are not. SOUND and COMBINATION are held side by side inside the instrument, yet their dump addresses are 786 432 apart; other pairs are the reverse.

Treat the address and length bytes together as a fixed name for one of the eight blocks. The instrument does the same — on reception it compares them against a fixed list and performs no arithmetic on them.

4.2 Data message

F0 50 2D 04 *nn* 11 *a2 a1 a0 l2 l1 l0 data... c sum* F7

nn unit field, 00 or 01.

a2 a1 a0 destination address, three bytes, most significant first, seven bits each: the value is $a_2 \times 128^2 + a_1 \times 128 + a_0$.

l2 l1 l0 length in *source* bytes, encoded the same way.

data the payload, encoded as described below.

c continuation flag: 01 if further messages follow, 00 on the last.

sum checksum.

A message is filled to at most 252 bytes before being sent; longer transfers are split.

4.3 Payload encoding

Each source byte is sent as two message bytes, **high nibble first**:

$$b \longrightarrow (b \gg 4), \quad (b \& 0F)$$

So source byte F1 appears on the wire as 0F 01, and the payload occupies twice as many bytes as the length field states. A receiver rejoins them as $((\text{first} \& 0F) \ll 4) | (\text{second} \& 0F)$.

An odd number of payload bytes is invalid.

4.4 Checksum

Let S be the eight-bit sum of every byte from the identifier 50 through the continuation flag inclusive. Then

$$\text{sum} = (0 - S) \& 7F$$

The leading F0, the checksum byte itself and the trailing F7 take no part in the sum. Equivalently, the sum of every byte after F0 including the checksum is zero modulo 128.

The checksum is not verified for every family

On reception the checksum is checked only for families 7E, 2B, 2C and 2D. For the other accepted families a wrong checksum passes unremarked.

4.5 Requesting a dump

A dump can be asked for over MIDI; the front-panel SEND button is not the only way to start one. The request names a category by its region byte:

F0 50 2B 04 *nn* 11 *area* 00 00 *x x x* F7

<i>area</i>	category requested
40	SYSTEM, PART & MIDI
20	SOUND
50	COMBINATION
60	SEQUENCER

The three bytes in the length position are ignored; any values are accepted. The instrument then runs exactly the transfer the corresponding menu row would have run.

There is no request for TOTAL KEYBOARD: the four categories must be asked for one at a time.

4.6 Opening a transfer

A dump begins with a two-step exchange. Each step is attempted up to three times.

1. The sender transmits F0 50 21 04 00 11 F7 and expects F0 50 22 04 *nn* 11 F7.
2. The sender transmits F0 50 22 04 00 11 F7 and expects F0 50 23 7E F7.

If the exchange succeeds, every subsequent data message must be acknowledged. If all three attempts of the first step fail, **the dump proceeds anyway, without acknowledgements**, at a fixed pace of roughly 51 ms between messages.

4.7 Acknowledgement and retry

With acknowledgements in force, each data message must be answered with F0 50 23 7E F7. Any other reply causes the same message to be retransmitted, up to three attempts, after which the transfer is abandoned.

A reply is awaited for about 5.1 seconds; during the opening exchange the wait is about 2.05 seconds.

4.8 Continuation

Data messages after the first open with F0 50 7E, followed by payload, continuation flag, checksum and F7.

4.9 Closing

At the end of each category the sender transmits F0 50 27 7E F7 and waits for an acknowledgement. At the end of the whole transfer it transmits F0 50 28 7E F7. If the transfer was abandoned after three failed retries it transmits F0 50 29 7E F7.

4.10 Receiving

When the SX-WSA1R is the receiver it answers each data message with `F0 50 23 7E F7` to accept, `F0 50 24 7E F7` to reject, and `F0 50 2A 7E F7` when the incoming data is larger than the destination.

While a bulk-dump session is open the instrument accepts only the `50` identifier; universal messages are not honoured until the session ends.

The instrument must be on the SYSEX BULK DUMP screen to receive

A dump cannot be pushed to an idle instrument. Reception happens only while that screen is displayed; a dump sent at any other time is parsed, discarded, and not reported.

A dump offered on the MIDI 2 input is never received either, at any time. This is what lies behind the published instruction to use MIDI 1 for bulk transfers.

Seven of the eight patterns are pinned on all six bytes, address and length together. Sending a correct address with a shortened length is not a short transfer — it is an unrecognised message, refused with ERROR 41. Only SEQUENCER block 3 has its length read as a number, because that block's size varies. An arbitrary destination cannot be written.

The one length the message itself controls is SEQUENCER part 3, which is capped at 330 752 bytes; beyond that, and whenever the destination fills mid-message, the instrument answers `F0 50 2A 7E F7` and reports ERROR 21.

Memory-full is treated as end-of-dump

When the SX-WSA1R is *sending* and the receiver answers `F0 50 2A 7E F7` to report that its memory is full, the instrument treats that exactly as it treats `F0 50 28 7E F7`, end of dump. It stops, and it reports success. A sender that needs to know whether the data arrived cannot rely on the instrument to notice.

Chapter 5

What a block contains

This reference describes the protocol that carries a dump. The meaning of the data inside a block is outside its scope, with one exception worth stating: every block opens with a fixed signature, and two of them carry readable names. Together these let a program check a stored dump before sending it back, and list what is in it, without understanding the rest.

5.1 Signatures

The bytes below are the first bytes of the reassembled block — after the payload has been rejoined from its nibbles, not as they appear on the wire.

block	first bytes	as text
SYSTEM, PART & MIDI 1	5A 5A 01 00 57 41 30 00	ZZ..WA0.
SYSTEM, PART & MIDI 2	78 10 ...	x.
SOUND	57 53 41 20 53 4F 55 4E	WSA SOUND RAM S0
COMBINATION 1	5A 5A 5A 5A 00 00 57 53	ZZZZ..WSA1
COMBINATION 2	78 10 ...	x.

A file whose first bytes do not match the block it claims to be is not a dump of that category.

5.2 Names

The SOUND block continues, immediately after its sixteen-byte signature, with sixteen-byte names in the instrument’s own character set, padded with spaces. The second COMBINATION block and the second SYSTEM, PART & MIDI block use a different framing: a two-byte prefix, then a sixteen-byte name, then two further bytes.

A librarian can therefore show the user what a stored file holds — “SWEEP PAD”, “Space Infinity” — without decoding anything else.

Reassembly is self-checking

A block reassembled correctly comes out at exactly the length its own header declared. A wrong nibble order, or a payload taken with its trailing flag and checksum still attached, does not. Checking the length is a cheap way to confirm a receiver is unpacking correctly before trusting anything it produces.

Chapter 6

Parameter messages

Two message families address individual parameters rather than transferring blocks. Family 2C writes; family 2B requests. Both share one address space.

F0 50 2B 04 *nn* 11 *r s p* ...F7

The three bytes after the model triple select what is addressed.

6.1 Parameter area

When *r* is 00, *s* selects a block and *p* selects a parameter within it.

<i>s</i>	block	parameters
00	common block 0	16
01	common block 1	6
08	write only	4
10	common block 10	20
11	common block 11	4, plus a further 68 values
20–3F	part 0 to part 31	57 each
60	single parameter	1

The part blocks are regular: *s* is 20 plus the part number, and all 32 parts take the same 57 parameters. A few parameters carry more than one data byte.

Under *s* = 11, values of *p* from 03 upward are admitted in three runs — 20–36, 40–56 and 60–75. Any other value in that block is accepted by the parser and then discarded, producing no error and no effect.

6.2 Whole-area request

Family 2B also requests an entire category, as described in chapter 4. Here *r* takes the region byte rather than 00.

6.3 A third region

Both families accept *r* values of 10, 18 and 19, which carry an address and a count. What they address is not covered by this edition.

Chapter 7

Universal System Exclusive

7.1 Recognised messages

The instrument honours two universal messages, and transmits both.

message	meaning
F0 7E 7F 09 01 F7	General MIDI System On
F0 7E 7F 09 02 F7	General MIDI System Off

Turning General MIDI on restricts the sounds and operations available and may rearrange the percussion map. General MIDI System Off has no effect if General MIDI is already off.

Both messages are also written into Standard MIDI Files produced by the instrument, as a five-byte System Exclusive event at delta time zero.

7.2 Messages not recognised

A controller expecting any of the following will receive no response:

message	
F0 7F 7F 04 01 <i>ll mm</i> F7	Master Volume
F0 7E <i>dd</i> 06 01 F7	Identity Request
F0 7E 7F 09 03 F7	General MIDI 2 System On

The instrument does not implement Identity Reply, so it cannot be auto-detected by a controller that relies on it. The entire Universal Real Time identifier 7F is unrecognised.

Chapter 8

Errors

Transfer failures are reported on the SYSEX BULK DUMP screen. A System Exclusive message arriving while any other screen is displayed produces no visible error.

8.1 The messages

screen	meaning
COMPLETED!	the transfer finished
ERROR 40!	the message is addressed to a different product
ERROR 41!	reception failed
ERROR 42!	transmission failed
ERROR 21!	the destination is full

Not every failure is a 40, 41 or 42

When incoming data is larger than the destination area, the instrument reports ERROR 21! MEMORY FULL. Software that recognises only the three documented System Exclusive errors will mis-read a truncated restore as something else.

8.2 ERROR 40 — wrong identification

Raised when the message family byte is accepted but the bytes that follow it do not match the product. For a message addressed to the SX-WSA1R this means the model triple was not 04, then 00 or 01, then 11.

8.3 ERROR 41 — reception

The general reception failure. It covers:

- a message inside a session that does not begin F0 50;
- a status byte other than F7 arriving inside an open message;
- a framing, parity or overrun error on the MIDI input, or loss of active sensing;
- no reply within the timeout;
- a message longer than 254 bytes;

- an odd number of payload bytes;
- a continuation flag that is neither 00 nor 01;
- a checksum mismatch.

8.4 ERROR 42 — transmission

Raised when the instrument cannot complete an outgoing transfer internally, roughly a second after the attempt stalls. In this version it has a single cause and does not indicate a fault on the MIDI cable or at the receiving device.

8.5 Interference from other traffic

While a bulk-dump session is open, unrelated System Exclusive traffic arriving on the MIDI input can set the error state, and will then be reported when the session ends. Keep other sources off the MIDI 1 input for the duration of a transfer.

Chapter 9

SX-WSA1 and SX-WSA1R

The SX-WSA1 keyboard and the SX-WSA1R rack module run the same program and speak the same System Exclusive protocol. Two features are nonetheless available on the keyboard and not on the rack, in both directions.

	SX-WSA1 (keyboard)	SX-WSA1R (rack)
message framing, checksum, handshake	yes	yes
SOUND bulk dump	yes	yes
COMBINATION bulk dump	yes	yes
SYSTEM, PART & MIDI bulk dump	yes	yes
dump requests	yes	yes
General MIDI On / Off	yes	yes
SEQUENCER bulk dump	yes	no
tempo message (family 25)	yes	no

On the rack, a SEQUENCER dump sends nothing and says nothing

The SEQUENCER row is present on the rack's SYSEX BULK DUMP menu, and selecting it and pressing SEND appears to work. Nothing is transmitted — not even the end-of-category message that closes every other transfer. A librarian waiting for data will simply time out.

The same applies to the SEQUENCER portion of TOTAL KEYBOARD.

The tempo message is refused in both directions on the rack: it will neither act on one received nor send one.

The model triple does not tell you which you are talking to

Both models transmit the same model triple, so it cannot be used to distinguish them. The only reliable test over MIDI is behavioural — request a SEQUENCER dump and see whether anything arrives.

Chapter 10

Relationship to other Technics instruments

The message engine described in this reference is not unique to the SX-WSA1R. At least two other Technics instruments implement the same protocol: the SX-KN5000 and the SX-KN1500.

10.1 What is shared

All three use the 50 identifier, the same message families, the same short control messages, the same payload encoding and the same checksum rule. The fixed messages are byte-for-byte identical:

```
F0 50 23 7E F7  F0 50 24 7E F7  F0 50 27 7E F7  F0 50 28 7E F7  F0 50 29 7E
F7  F0 50 2A 7E F7
```

The error mapping is also nearly common: of the thirty-four status values, thirty-two produce the same report on the SX-WSA1R and the SX-KN5000.

10.2 What distinguishes the products

The model triple is the discriminator. A message whose triple does not match the receiving instrument is refused with ERROR 40.

instrument	model triple
SX-WSA1R	04 00 11, 04 01 11
SX-KN5000	01 28 12
SX-KN1500	01 24 11, 01 25 11, 01 26 11

A librarian that supports more than one of these instruments needs only to vary the triple and the dump region addresses; the framing, handshake, encoding and checksum are common code.

10.3 Dump regions differ

The dump region addresses are per-product. The SX-KN5000 and SX-KN1500 share eleven of their thirteen region and length pairs with each other, but the SX-WSA1R shares none of its eight with either — its categories are its own.

The SX-WSA1R does not accept the GS-style messages its siblings do

The SX-KN5000 and SX-KN1500 additionally accept a small set of messages in the 42 12 parameter form, including the sequence 42 12 40 00 7F 00 41 used elsewhere in the industry as a reset. **The SX-WSA1R accepts none of them.** Software that resets a KN-series instrument that way and assumes the SX-WSA1R will follow will find it does not respond.

Chapter 11

Scope of this edition

This reference covers message framing, the bulk-dump transfer protocol, the universal messages, and error reporting. The following are not covered.

Parameter meanings. Chapter 6 gives the structure of the parameter address space and the size of each block, but not what each individual parameter controls.

Block contents. Chapter 5 gives each block's signature and where its names are, but not the meaning of the rest of the bytes — the structure of a sound, a combination or a sequence.

Operating system version 1. Everything here describes version 2.0.